

Relatório Final

Análise Léxica

Ferramenta utilizada

Utilizamos `ply.lex`, tal como pedido no enunciado, para implementar o analisador léxico do compilador.

Formato de código suportado

Optámos por tratar o código de entrada num formato próximo de **free-form**, em vez de implementar a lógica clássica de colunas fixas de Fortran 77.

Esta escolha simplifica a leitura do código, reduz a complexidade no lexer e ajusta-se melhor aos programas de teste usados no projeto, onde a estrutura relevante aparece organizada por palavras-chave, expressões e mudanças de linha.

Palavras-chave reconhecidas

Utilizamos um dicionário **reserved** para reconhecer diretamente palavras-chave da linguagem, incluindo PROGRAM, END, INTEGER, REAL, DOUBLE, PRECISION, COMPLEX, LOGICAL, CHARACTER, DIMENSION, PARAMETER, DATA, PRINT, READ, WRITE, IF, THEN, ELSE, ENDIF, DO, CONTINUE, GOTO, CALL, SUBROUTINE, FUNCTION, RETURN, MOD, STOP e PAUSE.

Utilizamos esta estrutura para centralizar todas as construções reservadas num único ponto e para distinguir automaticamente palavras da linguagem de identificadores definidos pelo utilizador.

Identificadores

Utilizamos um token ID para nomes de variáveis, arrays, funções, subrotinas e programas.

Convertimos todos os identificadores para maiúsculas no momento da análise léxica. Fazemo-lo para manter coerência com a natureza case-insensitive do Fortran e para evitar tratamento duplicado da mesma entidade nas fases seguintes.

Literais numéricos

Utilizamos `INT_CONST` para números inteiros e `REAL_CONST` para números reais.

Nos reais, suportamos representações decimais e notação exponencial com E, e, D ou d. A conversão de D para E é feita logo no lexer para uniformizar os valores antes de estes serem usados pelo parser e pela análise semântica.

Literais de texto e lógicos

Utilizamos `STRING_CONST` para cadeias entre aspas simples e tokens específicos para `.TRUE.` e `.FALSE.`.

Tratamos aspas simples duplicadas no interior de strings porque isso permite aceitar a forma habitual de escrita de texto em Fortran sem empurrar esse tratamento para fases posteriores.

Operadores utilizados

Utilizamos tokens para operadores aritméticos `+`, `-`, `*`, `/` e `**`, para o operador de atribuição `=`, para operadores relacionais `.EQ.`, `.NE.`, `.GT.`, `.GE.`, `.LT.` e `.LE.`, e para operadores lógicos `.AND.`, `.OR.` e `.NOT.`.

Esta separação permite que o parser receba diretamente cada categoria de operador já identificada, o que simplifica a definição de precedências e das regras gramaticais.

Símbolos especiais

Utilizamos tokens para `(`, `)`, `,`, `:` e para mudanças de linha com `NEWLINE`.

Incluímos estes símbolos porque são necessários para reconhecer listas de argumentos, acessos a arrays, declarações com dimensões, limites de intervalos e separação estrutural de instruções.

Palavras compostas normalizadas

Utilizamos regras léxicas específicas para reconhecer `END IF` e `GO TO`, convertendo-as respetivamente em `ENDIF` e `GOTO`.

Esta normalização evita espalhar casos especiais pela gramática e faz com que formas equivalentes da linguagem cheguem ao parser de modo uniforme.

Comentários e espaços ignorados

Ignoramos comentários introduzidos por `!`.

Também ignoramos espaços, tabulações e o carácter `\r`, preservando no entanto as mudanças de linha como tokens.

Mantemos `NEWLINE` porque a estrutura gramatical adotada no parser usa a separação por linhas para delimitar instruções.

Tratamento de erros

Utilizamos a exceção `LexError` para sinalizar caracteres ilegais, reportando o símbolo encontrado e a linha correspondente.

Esta opção ajuda a detetar cedo entradas inválidas e melhora a depuração durante os testes.

Dificuldades encontradas

Os principais pontos de atenção nesta fase foram o reconhecimento de formas compostas como `END IF` e `GO TO` e a necessidade de manter os identificadores normalizados para refletir o comportamento da linguagem.

Análise Sintática

Estrutura geral adotada

Utilizamos uma gramática escrita diretamente em funções Python, seguindo o modelo do `PLY`, e fazemos o parsing a partir dos tokens produzidos pelo lexer.

Esta abordagem permite manter numa única estrutura as regras da linguagem, a construção da árvore sintática e o tratamento de erros.

Estrutura de representação

Utilizamos uma classe `Node` para representar a árvore de sintaxe abstrata.

Cada nó guarda o tipo, os filhos, o valor associado e a linha de origem. Utilizamos esta estrutura para uniformizar a representação de programas, declarações, instruções e expressões, facilitando a análise semântica posterior.

Símbolo inicial e organização global

Utilizamos a regra `start` como ponto de entrada da gramática.

Esta regra aceita linhas em branco no início e no fim do ficheiro e suporta uma lista de unidades de programa, o que permite reconhecer um programa principal e também definições de `FUNCTION` e `SUBROUTINE`.

Unidades de programa

Utilizamos regras específicas para `PROGRAM`, `SUBROUTINE` e `FUNCTION`.

Incluímos estas três formas porque o enunciado exige suporte para o programa principal e considera `FUNCTION` e `SUBROUTINE` como valorização. Cada uma destas unidades é convertida para um nó próprio da AST.

Parâmetros formais

Utilizamos regras para listas de parâmetros e para listas opcionais vazias.

Isto permite tratar subrotinas e funções com ou sem argumentos, mantendo a representação uniforme na AST.

Lista de instruções

Utilizamos uma regra `statement_list` para agregar instruções e uma regra `statement_entry` para ligar cada instrução a uma quebra de linha.

Fazemo-lo porque a separação por linhas faz parte da estrutura escolhida para o parser e porque isso simplifica o reconhecimento sequencial das instruções.

Labels

Utilizamos `label_opt` para permitir labels inteiras opcionais antes de instruções.

Esta escolha é necessária para suportar `DO` com labels, `GOTO`, `computed GOTO` e outras construções de controlo de fluxo pedidas no enunciado.

Declarações suportadas

Utilizamos regras para declarações de `INTEGER`, `REAL`, `LOGICAL`, `CHARACTER`, `COMPLEX` e `DOUBLE PRECISION`.

Também suportamos listas de identificadores e declaração direta de arrays com dimensões. Isto cobre a parte do enunciado relativa à declaração de tipos e variáveis.

Arrays, dimensões e dados declarativos

Utilizamos regras para `DIMENSION`, `PARAMETER` e `DATA`.

Incluimos estas construções porque aparecem como elementos relevantes da linguagem e porque permitem descrever arrays, constantes simbólicas e inicialização declarativa dentro da mesma gramática.

Instruções de atribuição e acesso

Utilizamos regras para atribuição simples e para atribuição a posições de arrays.

Esta separação permite distinguir logo na AST entre escrita sobre variáveis escalares e escrita sobre arrays.

Input e output

Utilizamos regras para `READ`, `PRINT` e `WRITE`.

Incluimos estas instruções porque o enunciado exige operações básicas de input e output. Suportamos tanto formas simplificadas com `*` como formas com unidade e formato explícitos.

Condicionais

Utilizamos regras para três formas de `IF`: `IF-THEN[-ELSE]-ENDIF`, `logical IF` de uma única instrução e `arithmetic IF`.

Esta divisão permite cobrir as formas condicionais mais relevantes presentes nos exemplos do enunciado e separa logo estruturalmente variantes que depois exigem validações semânticas diferentes.

Controlo de fluxo

Utilizamos regras para `DO`, `CONTINUE`, `GOTO` e `computed GOTO`.

Incluímos estas instruções porque fazem parte explícita dos requisitos técnicos do projeto, em particular os ciclos `DO` com labels e os desvios com `GOTO`.

Chamadas e retorno

Utilizamos regras para `CALL` e `RETURN`.

Estas construções são necessárias para suportar subprogramas e encaixam na valorização pedida no enunciado para `FUNCTION` e `SUBROUTINE`.

Expressões

Utilizamos regras para identificadores, literais, chamadas, acessos com argumentos, slices e operadores aritméticos, relacionais e lógicos.

Esta organização permite representar de forma uniforme expressões usadas em atribuições, condições, limites de ciclos e argumentos de chamadas.

Precedência e associatividade

Utilizamos uma tabela de precedência para `OR`, `AND`, `NOT`, operadores relacionais, operadores aritméticos e `UMINUS`.

Fazemo-lo para reduzir ambiguidades e para garantir que a AST respeita a ordem de avaliação esperada nas expressões.

Tratamento de erros

Utilizamos a exceção `ParseError` para sinalizar erros sintáticos, indicando o símbolo problemático e a linha correspondente, ou o fim do ficheiro quando necessário.

Esta abordagem facilita a localização de erros durante os testes e durante a integração com as restantes fases do compilador.

Exemplos

Exemplo de programa simples:

```
PROGRAM TEST
INTEGER X
X = 1
END
```

AST simplificada produzida para este exemplo:

```
MainProgram(TEST)
|-- Statement
|   |-- Declaration(INTEGER)
|       |-- ID(X)
|-- Statement
    |-- Assignment(X)
        |-- Literal(1)
```

Exemplo com ciclo e label:

```
PROGRAM SOMA
INTEGER I
DO 10 I = 1, 5
10 CONTINUE
END
```

AST simplificada produzida para este exemplo:

```
MainProgram(SOMA)
|-- Statement
|   |-- Declaration(INTEGER)
|       |-- ID(I)
|-- Statement
|   |-- Do(label=10, var=I)
|       |-- Literal(1)
|       |-- Literal(5)
|-- Statement(label=10)
    |-- Continue
```

Dificuldades encontradas

Os principais pontos de atenção nesta fase foram a organização da gramática em torno de `NEWLINE`, a coexistência de várias formas de `IF`, a distinção entre chamada de função e acesso a array, e a representação coerente das instruções com labels na AST.

Análise Semântica

Estrutura geral adotada

Organizamos a análise semântica em três componentes: `symbol_table.py`, `semantic_table_builder.py` e `semantic.py`.

Esta divisão separa a representação da tabela de símbolos, a recolha inicial de informação a partir da AST e a fase de validação semântica propriamente dita.

Tabela de símbolos

Utilizamos as classes `Symbol` e `SymbolTable` para guardar a informação semântica relevante.

Guardamos símbolos globais e locais, labels, referências a labels e scopes filhos. Esta estrutura permite validar nomes, tipos, arrays, funções, subrotinas e controlo de fluxo de forma consistente.

Estrutura dos símbolos

Cada símbolo guarda o nome, a categoria, o tipo, as dimensões, os parâmetros e o tipo de retorno, quando aplicável.

Utilizamos esta organização para distinguir programas, variáveis, arrays, parâmetros, funções, subrotinas e intrínsecas sem depender apenas do nome do identificador.

Scopes

Utilizamos um scope global e um scope filho para cada unidade de programa.

Esta separação permite validar de forma independente `PROGRAM`, `FUNCTION` e `SUBROUTINE`, preservando a visibilidade correta dos símbolos e evitando conflitos indevidos entre unidades diferentes.

Exemplo simplificado da organização dos scopes:

```
global
`-- MAIN
    |-- X : variable, INTEGER
    |-- A : array, REAL
    `-- label 10
```

Construção inicial da informação semântica

Utilizamos o `SymbolTableBuilder` para percorrer a AST antes da validação principal.

Nesta fase recolhemos unidades globais, criamos scopes, registamos parâmetros formais, declarações de variáveis e arrays, labels definidas e referências a labels usadas por `DO` e `GOTO`.

Percurso da AST

Utilizamos a AST como estrutura base para recolher e validar informação semântica.

Numa primeira passagem, percorremos a AST com o `SymbolTableBuilder` para guardar os dados estruturais necessários na tabela de símbolos, nomeadamente

unidades de programa, scopes, parâmetros, declarações, labels e referências a labels.

Numa segunda passagem, percorremos novamente a AST com o **SemanticValidator** para validar usos concretos dos símbolos, compatibilidade de tipos, inicialização de variáveis, chamadas, arrays e controlo de fluxo.

Esta separação entre recolha e validação simplifica a implementação, porque quando uma instrução é validada a informação relevante sobre símbolos e scopes já está disponível.

Estrutura global do programa

Validamos que existe pelo menos uma unidade de programa e que existe no máximo um **PROGRAM** principal.

Fazemo-lo logo na fase de construção da tabela de símbolos para detetar cedo erros estruturais do programa.

Identificadores e declarações

Utilizamos validações para garantir que os símbolos são declarados antes de serem usados e que não existem declarações repetidas no mesmo scope.

Também validamos a diferença entre variáveis escalares, arrays, funções e sub-rotinas, para impedir usos semanticamente incorretos do mesmo identificador.

Ordem entre declarações e instruções executáveis

Utilizamos um controlo explícito da secção declarativa através da variável `seen_executable_statement`.

Isto permite impor a regra de que declarações, **DIMENSION**, **PARAMETER** e **DATA** devem aparecer antes das instruções executáveis dentro da unidade de programa.

Inicialização de variáveis

Utilizamos um conjunto de símbolos inicializados para controlar se uma variável já recebeu valor antes de ser usada em expressões.

Esta verificação permite detetar usos de variáveis declaradas mas ainda não inicializadas. Também consideramos que **READ** inicializa as variáveis que recebe.

Tipos de dados

Utilizamos a inferência de tipos para literais, identificadores, acessos a arrays, chamadas, operadores binários e operadores unários.

Com essa informação validamos compatibilidade em atribuições, condições, operadores aritméticos, operadores relacionais e operadores lógicos.

Arrays

Utilizamos validações específicas para arrays, incluindo verificação da aridade e do tipo dos índices.

Esta separação permite impedir que uma variável escalar seja usada como array, que um array seja usado como escalar e que o número de índices não corresponda ao número de dimensões declaradas.

Input e output

Utilizamos validações para `READ`, `PRINT` e `WRITE`.

Em `READ`, aceitamos apenas variáveis ou posições de arrays como destinos. Em `PRINT` e `WRITE`, validamos semanticamente as expressões fornecidas.

Condicionais

Utilizamos validações distintas para `If`, `LogicalIf` e `ArithmeticIf`.

Nos `IF` lógicos exigimos condições do tipo `LOGICAL`. No `ArithmeticIf`, exigimos uma expressão numérica e validamos as labels associadas.

Controlo de fluxo

Utilizamos a recolha e validação de labels para `DO`, `GOTO` e `computed GOTO`.

Verificamos se a label existe, se o `DO` aponta para uma label válida e se a label final do ciclo corresponde a uma instrução `CONTINUE`, de acordo com o que o enunciado pede explicitamente.

Funções, subrotinas e intrínsecas

Utilizamos símbolos próprios para `FUNCTION`, `SUBROUTINE` e para a intrínseca `MOD`.

Também validamos a aridade das chamadas, a compatibilidade entre argumentos reais e parâmetros formais e, no caso das funções, se o nome da função recebe valor dentro do respetivo scope.

Tratamento de erros

Utilizamos a exceção `SemanticError` para sinalizar erros semânticos.

Em vez de parar no primeiro problema, acumulamos vários erros ao longo da validação quando isso é possível. Isto melhora bastante a fase de testes e a análise de programas incorretos.

Exemplo

Exemplo simples de informação semântica recolhida para um programa com declaração, atribuição e ciclo:

```
global
`-- SOMA
    |-- I : variable, INTEGER
    |-- labels: [10]
    |-- label_references:
        |-- DO -> 10
```

Dificuldades encontradas

Os principais pontos de atenção nesta fase foram a distinção entre declaração e inicialização, a validação coerente de arrays e chamadas, e a verificação correta de labels em `DO` e `GOTO` sem perder a separação por scopes.